

Labwork 3: Hello, CUDA!

Vu Ha Vy – 2540112

September 29, 2026

1 CPU vs. GPU Performance

In this experiment, the input image was processed on both CPU and GPU.



Figure 1: Input Image

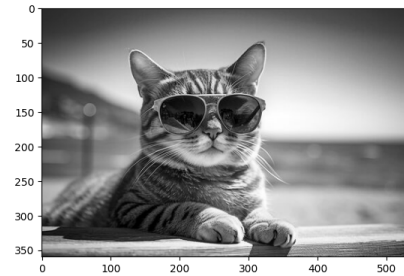


Figure 2: Output Image

CPU Implementation:

The CPU processes pixels sequentially through a loop, calculating the average RGB value to produce a grayscale pixel and assigning it to all three color channels. It then calls this function on the flattened image and reshapes the output array back into the original 2D image dimensions.

```
for i in range(len(src)):
    g = np.uint8((int(src[i,0]) + int(src[i,1]) + int(src[i,2])) / 3)
    dst[i,0] = dst[i,1] = dst[i,2] = g
cpuDst = grayscale_cpu(flatImg)
cpuImg = cpuDst.reshape(height, width, 3)
```

GPU Implementation:

The GPU implementation processes pixels concurrently using parallel CUDA threads, where each thread identifies its target element using a global index derived from `cuda.threadIdx.x`, `cuda.blockIdx.x`, and `cuda.blockDim.x`. Data is transferred from host memory to device memory using `cuda.to_device()`.

before launching the `@cuda.jit` kernel across the grid. Once parallel execution completes on the GPU, the result is transferred back to system memory via `copy_to_host()` and reshaped into the final image dimensions.

```

tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
g = np.uint8((src[tidx, 0] + src[tidx, 1] + src[tidx, 2]) / 3)
dst[tidx, 0] = dst[tidx, 1] = dst[tidx, 2] = g
devSrc = cuda.to_device(flatImg)
devDst = cuda.device_array(flatImg.shape, np.uint8)
grayscale[gridSize, blockSize](devSrc, devDst)
hostDst = devDst.copy_to_host()
gpuImg = hostDst.reshape(height, width, 3)

```

Running time

Implementation	Running Time (s)
CPU	0.3203566074371338
GPU	0.09885144233703613

Table 1: Running Time Comparison

Conclusion: The GPU implementation had a speedup of approximately 3.24x compared to the CPU implementation, reducing the processing time from 0.320 seconds to 0.099 seconds.

2 Impact of CUDA Block Size

Block size	Running Time (s)
32	0.004130840301513672
64	0.0010914802551269531
128	0.003341197967529297
256	0.0018801689147949219
512	0.0017004013061523438

Table 2: Execution Time for Different Block Sizes

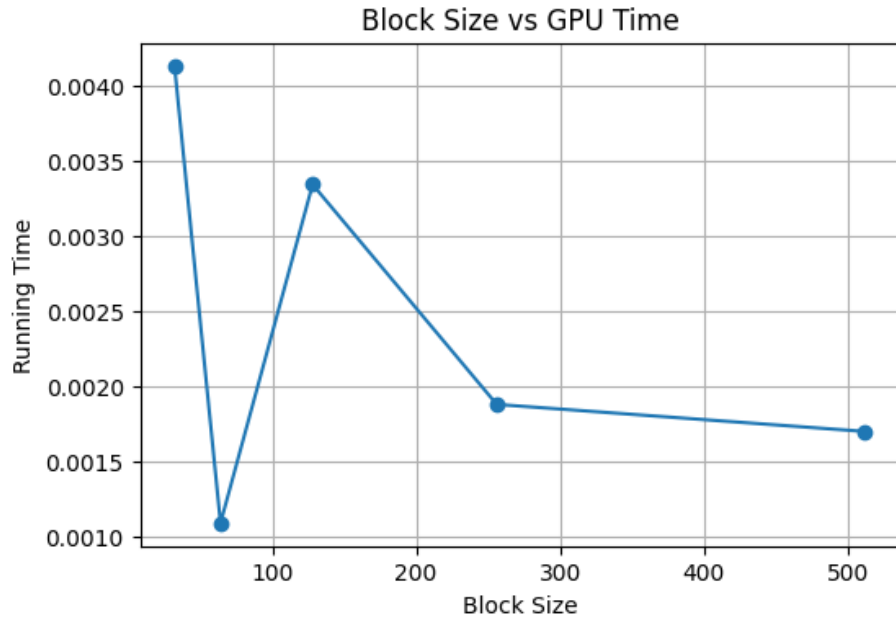


Figure 3: Running Time Across Different CUDA Block Sizes

Pattern Analysis:

The execution time decreases significantly from a block size of 32 to 64, reaching the fastest performance at 64 (approx. 0.00109 seconds). However, increasing the block size to 128 caused a slowdown, and then the performance leveled off around 0.0017s - 0.0018s for 256 and 512. This shows that increasing block size does not strictly increase the performance. It also relies on balancing thread occupancy and hardware resource allocation per streaming multiprocessor.